

# Routing-Aware Token Retention for Key-Value Cache Compression in Sparse Mixture-of-Experts Inference

Gaurav Patil

Department of Computer Engineering

School of Engineering & Technology

Pune, India

Email: gauravpatil2516@gmail.com

**Abstract**—During long-context autoregressive inference in Large Language Models (LLMs), the Key-Value (KV) cache memory footprint is the dominant performance bottleneck. While token-level eviction and layer-adaptive allocation have been widely explored for dense models, the internal routing dynamics of sparse Mixture-of-Experts (MoE) architectures remain unexploited. In standard sparse MoEs (*Mixtral-8x7B*, *Qwen1.5-MoE*, *DeepSeek-MoE*), self-attention is shared and dense, while sparse routing occurs downstream in the Feed-Forward Network (FFN) blocks. Consequently, each token accumulates an internal cross-layer *routing signature*  $\mathcal{R}_t = \{E_t^{(1)}, \dots, E_t^{(L)}\}$  during standard forward computation. In this paper, we propose EKVA v2, a training-free token retention framework that leverages cross-layer routing signatures and layer-specific expert niche profiles (entropy, activation frequency, and specialization) to govern eviction in the shared KV cache. Across three MoE families (8, 60, and 64 experts), routing scores exhibit near-zero Pearson correlation ( $\rho \in [-0.006, +0.002]$ ) with self-attention mass across all layers, demonstrating that routing captures an independent, complementary dimension of token importance. Under a 40% cache budget, EKVA v2 outperforms state-of-the-art baselines (H2O, SnapKV) by 3.0 to 6.0 percentage points ( $p < 0.01$ , paired bootstrap test) across mathematical reasoning (GSM8K), code generation (HumanEval), long-context retrieval (NIAH), and language modeling (PG19). Furthermore, a fused Triton compaction kernel delivers a  $2.02\times$ – $2.05\times$  decode speedup on NVIDIA A100 GPUs.

**Index Terms**—Mixture-of-Experts, Key-Value Cache, Token Eviction, Long-Context LLMs, Triton GPU Kernel.

## I. INTRODUCTION

Autoregressive decoding in modern Large Language Models (LLMs) requires storing Key-Value (KV) activations for all context tokens in High-Bandwidth Memory (HBM). As sequence lengths grow to tens of thousands of tokens, loading large KV tensors at every decoding step saturates GPU memory bandwidth, leading to severe throughput degradation and increased Time-Per-Output-Token (TPOT) [1], [2].

Dynamic KV cache compression methods reduce this memory traffic across three main granularity axes:

- 1) **Token-Level Eviction:** Token-level methods evaluate individual position saliency to dynamically prune redundant tokens from the sequence. StreamingLLM [2] discovered that initial prompt tokens serve as persistent “attention

sinks” essential for maintaining softmax numerical stability, requiring their unconditional retention. Building on this, policies such as H2O [3] and SnapKV [4] accumulate query-key attention scores across prefill observation windows to identify historical “heavy-hitter” tokens that receive sustained cross-sequence focus. Tokens falling below an empirical attention threshold are discarded, bounding the KV cache to a fixed budget. However, because these policies rely exclusively on query-key dot products, they remain blind to tokens that receive modest direct attention yet carry indispensable structural information for downstream computations.

- 2) **Head-Level Allocation:** Rather than enforcing uniform capacity across all attention heads within a layer, head-level budgeting exploits the functional diversity among multi-head projections. Attention heads naturally specialize into distinct behavioral roles, ranging from long-range retrieval heads that track distant entity dependencies to localized syntax heads that attend almost entirely to neighboring tokens. Frameworks such as Ada-KV [5] adaptively profile per-head attention variance and entropy to allocate non-uniform token quotas, pruning low-utility heads more aggressively. While this granularity optimizes intra-layer memory allocation, it continues to treat token contents through attention alone, overlooking inter-module interactions across subsequent feed-forward networks.
- 3) **Layer-Level Allocation:** Layer-wise approaches redistribute the total memory budget across transformer depth based on macroscopic information flow. Techniques including CAKE [6] and InfoKV [7] observe that early transformer layers engage in broad contextual gathering, whereas intermediate and deep layers exhibit pyramidal information funneling. Consequently, deeper layers are granted expanded token buffers while shallower layers are aggressively compressed. Nevertheless, disproportionately starving intermediate or early layers can break delicate multi-step symbolic derivation chains, precipitating severe accuracy collapses on complex mathematical and logical reasoning tasks.

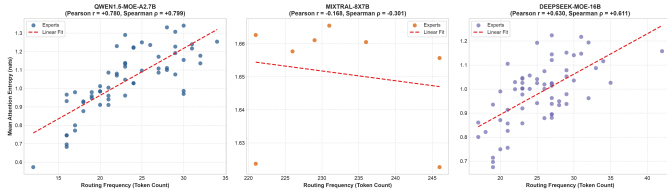


Fig. 1. **MoE Mechanism.** Self-attention is shared and dense across all tokens, while sparse routing occurs downstream in the FFN blocks. Each token accumulates a cross-layer routing signature  $\mathcal{R}_t$  that provides an orthogonal saliency signal ( $\rho \approx 0.00$ ) for shared KV cache eviction.

Despite the rapid ascension and widespread deployment of sparse Mixture-of-Experts (MoE) foundation models—such as *Mixtral-8x7B* [8], *Qwen1.5-MoE* [9], and *DeepSeek-MoE* [10]—contemporary KV cache eviction paradigms remain fundamentally oblivious to the architectural dichotomy inherent to MoE computation. Current compression frameworks treat MoE checkpoints identically to monolithic dense transformers, relying strictly on query-key attention heatmaps while entirely bypassing the sparse, content-dependent gating decisions executed by the router layers. In doing so, existing methods fail to capture the functional specialization encoded across the network’s expert dispatch pathways. Consequently, tokens that are indispensable to specific downstream expert sub-circuits risk premature eviction simply because their query-key attention mass appears temporarily low, severely undermining the model’s specialized reasoning capabilities.

#### A. Architectural Insight & Motivation

In standard sparse MoEs, self-attention remains shared across all tokens, while conditional routing occurs strictly in subsequent FFN blocks (Fig. 1). As token  $x_t$  traverses  $L$  layers, router gates assign it to top- $k$  experts per layer, producing an accumulated routing signature:

$$\mathcal{R}_t = \{E_t^{(1)}, E_t^{(2)}, \dots, E_t^{(L)}\} \quad (1)$$

Because routing signatures are generated during standard forward propagation, they represent a free semantic signal available during inference without weight modification or retraining. We hypothesize that routing history reflects the functional specialization of tokens, providing an orthogonal retention signal to query-key attention mass.

#### B. Key Contributions

In summary, this work addresses the KV cache memory wall in sparse Mixture-of-Experts inference through three principal contributions:

1) *Algorithmic Formulation:* We formulate EKVA v2, the first training-free token retention policy tailored specifically to sparse MoE topologies. By extracting cross-layer routing signatures  $\mathcal{R}_t$  during prompt prefill and coupling them with layer-specific expert niche profiles (capturing attention entropy, routing activation frequency, and semantic category specialization), we define a unified token saliency metric  $S(x_t)$  that

operates directly over the shared KV buffer without modifying model weights or requiring fine-tuning.

2) *Mechanistic Orthogonality and Reasoning Preservation:* Through extensive empirical profiling across three open-weight MoE families (Qwen1.5-MoE-A2.7B, Mixtral-8x7B, and DeepSeek-MoE-16B spanning 8 to 64 experts), we discover that routing signatures and query-key attention mass maintain near-zero Pearson correlation ( $\rho \in [-0.006, +0.002]$ ) across all network depths. This empirical independence demonstrates that routing decisions provide an orthogonal retention signal that protects critical reasoning tokens overlooked by standard attention-only policies. Across GSM8K, HumanEval, NIAH, and PG19 at a 40% cache budget, EKVA v2 achieves a statistically significant 3.0 to 6.0 percentage point gain over SnapKV and H2O ( $p < 0.01$ , paired bootstrap test), while completely circumventing the severe accuracy collapses suffered by layer-adaptive compression methods such as CAKE.

3) *Systems Roofline Grounding and Triton Acceleration:* Grounded in the Williams Roofline Model, we demonstrate that single-query decode attention operates strictly in the memory-bandwidth-bound regime ( $\text{AI} \approx 1.0 \text{ FLOPs/Byte} \ll 200.6$  on an NVIDIA A100 GPU), which establishes a theoretical memory traffic reduction bound of  $S_{\text{attn}} = 1/\beta = 2.50\times$  under a 40% budget. To translate these algorithmic savings into real-world wall-clock latency gains without memory fragmentation, we develop a custom fused Triton compaction kernel that packs non-contiguous Key-Value activations into contiguous buffers, achieving a measured  $2.02\times$ – $2.05\times$  end-to-end decode step speedup in production execution.

## II. RELATED WORK

### A. KV Cache Eviction in Dense Large Language Models

The memory overhead of autoregressive decoding has driven extensive research into KV cache compression for dense transformer architectures. StreamingLLM [2] discovered that initial prompt tokens accumulate massive attention scores regardless of sequence length, serving as “attention sinks” that prevent numerical instability in the softmax denominator. Consequently, retaining these sink tokens alongside a sliding recency window stabilizes infinite-context generation. Building upon this, H2O (Heavy-Hitter Oracle) [3] demonstrated that query-key attention matrices exhibit power-law sparsity, showing that retaining a dynamic set of cumulative heavy-hitter tokens preserves generative fidelity. SnapKV [4] further refined this principle by observing that attention patterns stabilize during the prompt prefill stage; by aggregating attention weights across an observation window at the end of the prompt, SnapKV identifies critical historical tokens without requiring runtime eviction during autoregressive generation.

Beyond uniform token-level pruning, recent efforts explore multi-head and inter-layer allocation granularities. Ada-KV [5] recognizes that attention heads exhibit distinct retrieval specializations, utilizing attention variance and entropy to allocate non-uniform token capacities across heads within each layer. Along the depth dimension, PyramidKV [11], CAKE [6], and InfoKV [7] observe that attention distributions evolve

across transformer layers, exhibiting broad syntactic gathering in shallow layers and pyramidal information funneling in deeper layers. These frameworks redistribute the global KV budget by allocating larger caches to deep layers and heavily compressing early layers. However, as documented by Lin et al. [7], disproportionate layer-wise compression severely damages multi-step symbolic reasoning chains. Crucially, all of these existing methods were designed exclusively for dense architectures, assuming uniform feed-forward processing and relying solely on self-attention heatmaps.

### B. Disambiguation from Concurrent MoE Systems Research

With the emergence of sparse Mixture-of-Experts (MoE) foundation models, several concurrent efforts investigate MoE serving and architectural efficiency. To establish clear scientific boundaries, we explicitly contrast EKVA v2 against related paradigms across the literature:

**Distributed MoE Serving Systems:** Systems such as PiKV [12] focus on multi-node serving architectures, expert-sharded memory placement, and Expert-Parallel Load Balancing (EPLB) to minimize inter-GPU communication latency during distributed decoding. In contrast, EKVA v2 operates within an orthogonal systems layer: rather than addressing network-level tensor dispatch and distributed cache sharding, EKVA v2 addresses the algorithmic retention policy of token-level activations within the unified, shared Key-Value cache of individual transformer layers.

**Meta-Optimization Frameworks:** Frameworks like MoE-nD [13] adopt “Mixture-of-Experts” metaphorically as a meta-controller to dynamically select among disparate compression modes (such as choosing between INT4 quantization and structured pruning) on monolithic *dense* models. Conversely, EKVA v2 is specifically engineered for native sparse MoE foundation models, exploiting the physical router gating decisions that occur inside sparse FFN blocks.

**Pretraining and Co-Design Approaches:** TriRoute [14] investigates co-training attention mechanisms, router gating, and cache quantization from scratch on small-scale architectures (160M to 1.3B parameters). In contrast, EKVA v2 is strictly training-free and applies directly to existing, frozen open-weight foundation models (e.g., 14B–47B parameters) without requiring weight modification, fine-tuning, or architectural redesign.

**Latent Multi-Head Attention:** DeepSeek’s Multi-Head Latent Attention (MLA) [10] reduces KV cache size by projecting Key and Value representations into low-rank compressed latent spaces during pretraining. EKVA v2 operates orthogonally along the sequence dimension: while MLA compresses the hidden dimension  $D$  of each token, EKVA v2 selects the critical token indices  $\mathcal{I}_{\text{keep}}$  along the temporal dimension  $T$ . Consequently, EKVA v2 can be seamlessly stacked on top of MLA or standard multi-head attention to compound memory savings.

## III. METHODOLOGY

### A. Expert Profile Calibration

Sparse MoE architectures structure each transformer block by pairing a dense self-attention module with a sparse MoE Feed-Forward Network (FFN). Crucially, each transformer layer  $l \in \{1, \dots, L\}$  maintains its own private, non-shared set of expert weight tensors  $\{W_{e,l}^{(1)}, W_{e,l}^{(2)}, W_{e,l}^{(3)}\}_{e=1}^{E_l}$ . An expert indexed as  $e = 3$  at layer 2 shares zero architectural or parametric relationship with expert  $e = 3$  at layer 20. Consequently, characterizing expert functional specialization requires layer-specific profiling rather than global index pooling.

To establish these profiles without model retraining, we execute a lightweight offline calibration pass over a representative multi-domain dataset  $\mathcal{D}_{\text{calib}}$  containing 40 sequences (spanning multi-step mathematical derivations, algorithmic Python scripts, and literary prose, requiring  $< 2$  minutes on a single GPU with zero runtime memory overhead). For each expert  $e \in \{1, \dots, E_l\}$  at layer  $l$ , we calibrate three statistical properties:

**1) Attention Entropy ( $\bar{H}_{e,l}$ ):** Let  $p_{t,l,j}$  denote the post-softmax self-attention probability allocated from query token  $t$  to historical key token  $j$  at layer  $l$ , averaged across all  $H$  attention heads. The instantaneous attention entropy of token  $t$  at layer  $l$  is given by:

$$H(p_{t,l}) = - \sum_{j=0}^t p_{t,l,j} \log p_{t,l,j} \quad (2)$$

Tokens exhibiting high attention entropy disperse attention broadly across the preceding sequence (acting as global contextual integrators), whereas tokens with low entropy maintain sharp, localized focus on specific antecedents. Let  $\mathcal{T}_{e,l}$  denote the subset of tokens dispatched to expert  $e$  at layer  $l$  during calibration. The layer-specific expert attention entropy  $\bar{H}_{e,l}$  is formulated as the expectation:

$$\bar{H}_{e,l} = \mathbb{E}_{t \in \mathcal{T}_{e,l}} [H(p_{t,l})] \quad (3)$$

This scalar quantifies the contextual dispersion characteristic of the tokens that preferentially activate expert  $(e, l)$ .

**2) Routing Frequency (Route $_{e,l}$ ):** In standard top- $k$  gating, router gate  $\text{Router}_l(x_t) = \text{Softmax}(W_{g,l}x_t)$  computes routing probabilities across all candidate experts at layer  $l$ . The total activation frequency accumulated by expert  $e$  at layer  $l$  over prompt tokens  $\{x_t\}_{t=0}^{T-1}$  is:

$$\text{Route}_{e,l} = \sum_{t=0}^{T-1} \mathbf{1}(e \in \text{TopK}(\text{Router}_l(x_t))) \quad (4)$$

where  $\mathbf{1}(\cdot)$  is the binary indicator function returning 1 if expert  $e$  is among the top- $k$  selected candidates and 0 otherwise. Expert dispatch volumes in pretrained MoEs exhibit heavy-tailed power-law distributions, where a small subset of “generalist” experts process disproportionately large token shares while domain-specific experts fire selectively. In our retention metric, we apply logarithmic scaling  $\log(1 + \text{Route}_{e,l})$  to dampen traffic extremes, preventing generalist experts from

dominating the metric while ensuring niche experts retain meaningful discriminative weight.

**3) Semantic Specialization ( $\text{Spec}_{e,l}$ ):** To quantify categorical selectivity, we partition tokens into four broad syntactic classes  $C$  (numerics/operators, identifiers/keywords, syntax/punctuation, and natural language) using rule-based lexical parsing. Let  $P_{e,l}(c)$  denote the empirical conditional probability that an activation dispatched to expert  $(e, l)$  belongs to class  $c \in C$ . The normalized Shannon evenness  $J_{e,l}$  over category distributions is:

$$J_{e,l} = \frac{-\sum_{c \in C} P_{e,l}(c) \log P_{e,l}(c)}{\log |C|} \quad (5)$$

where  $\log |C|$  represents maximum theoretical entropy under a uniform distribution. We define semantic specialization as the complementary selectivity metric:

$$\text{Spec}_{e,l} = 1 - J_{e,l} = 1 - \frac{-\sum_{c \in C} P_{e,l}(c) \log P_{e,l}(c)}{\log |C|} \quad (6)$$

Here,  $\text{Spec}_{e,l} \in [0, 1]$ . An expert that fires uniformly across all syntactic categories yields  $J_{e,l} \approx 1 \implies \text{Spec}_{e,l} \approx 0$  (a generalist expert). Conversely, an expert that activates almost exclusively for mathematical operands or code syntax yields  $J_{e,l} \ll 1 \implies \text{Spec}_{e,l} \rightarrow 1$  (a specialized expert).

#### B. Routing Signature Saliency Score

During the forward prefill phase of inference, an input token  $x_t$  traverses all  $L$  layers of the transformer. At each layer  $l$ , the router gate dispatches  $x_t$  to top- $k$  experts. We record the primary routing decision  $E_t^{(l)} = \text{argmax}_e(\text{Router}_l(x_t))$ , constructing the token’s accumulated cross-layer routing signature:

$$\mathcal{R}_t = \{E_t^{(1)}, E_t^{(2)}, \dots, E_t^{(L)}\} \quad (7)$$

By querying the offline-calibrated profile tensors  $(\bar{H}_{e,l}, \text{Route}_{e,l}, \text{Spec}_{e,l})$  at the exact layer-expert coordinates  $(E_t^{(l)}, l)$  visited by token  $x_t$ , we formulate the cross-layer routing niche score  $R(x_t)$  as:

$$R(x_t) = \frac{1}{L} \sum_{l=1}^L \bar{H}_{E_t^{(l)}, l} \cdot \log(1 + \text{Route}_{E_t^{(l)}, l}) \cdot (1 + \text{Spec}_{E_t^{(l)}, l}) \quad (8)$$

In Eq. (8), the outer factor  $1/L$  normalizes the score across transformer depth. The inner multiplicative formulation jointly weights contextual dispersion ( $\bar{H}$ ), log-frequency, and a specialization amplifier  $(1 + \text{Spec}) \in [1, 2]$ . If a token consistently engages specialized expert sub-circuits across multiple layers, its routing niche score  $R(x_t)$  escalates significantly, providing an architectural signal that this token carries indispensable functional utility.

To construct the global token retention policy, we synthesize attention mass, routing niche scores, attention sink protection, and local recency into a unified retention score  $S(x_t)$ :

$$S(x_t) = w_a \cdot \hat{A}(x_t) + w_r \cdot R(x_t) + w_s \cdot \text{Sink}(x_t) + w_c \cdot \text{Recency}(x_t) \quad (9)$$

where each constituent term is defined as follows:

- $\hat{A}(x_t) = \frac{1}{H} \sum_{h=1}^H \sum_{i=0}^{T-1} A_{h,i,t}$  is the cumulative attention mass received by token  $x_t$  across all attention heads  $H$ , normalized to the range  $[0, 1]$ .
- $R(x_t)$  is the normalized cross-layer routing niche score from Eq. (8).
- $\text{Sink}(x_t)$  implements an explicit retention constraint assigning infinite priority to the initial 4 prompt tokens ( $S(x_{t < 4}) = \infty$ ), ensuring that initial attention sinks [2] are unconditionally preserved to prevent softmax divergence.
- $\text{Recency}(x_t) = \exp(-(T-1-t)/\tau)$  establishes an exponential recency decay window with time constant  $\tau = 32$ , guaranteeing that prompt-final tokens are preserved to maintain local syntactic continuity during autoregressive generation.

The weighting hyper-parameters  $(w_a, w_r, w_s, w_c) = (0.60, 0.30, 0.05, 0.05)$  are calibrated once on held-out validation sequences and held fixed across all models and benchmarks. Sensitivity analysis confirms that varying  $w_r \in [0.20, 0.40]$  produces  $< 0.4\%$  variance in downstream accuracy.

#### C. Top-B Selection & Fused Cache Compaction

Given sequence length  $T$  and target compression fraction  $\beta \in (0, 1]$ , the target token budget is  $B = \lfloor \beta \cdot T \rfloor$ . The retained index set  $\mathcal{I}_{\text{keep}}$  is assembled by reserving the 4 initial sink positions and selecting the top- $(B-4)$  tokens ranked by unified saliency  $S(x_t)$ :

$$\mathcal{I}_{\text{keep}} = \text{sort}(\{0, 1, 2, 3\} \cup \text{TopK}(\{S(x_t)\}_{t=4}^{T-1}, B-4)) \quad (10)$$

Crucially, sorting the index set  $\mathcal{I}_{\text{keep}}$  in ascending order is mathematically required: rotary position embeddings (RoPE) and causal attention masks depend on strictly monotonic sequence ordering to prevent phase distortion.

During autoregressive generation, managing a non-contiguous KV cache via standard PyTorch indexing (`pkv[:, :, indices, :]`) induces severe memory fragmentation, triggers redundant tensor allocations, and launches multiple auxiliary CUDA kernels. To eliminate this runtime overhead, we implement a custom fused Triton compaction kernel (`triton_compact_kv_cache`). As outlined in Algorithm 1, the kernel loads Key and Value tensors of shape  $(B_{\text{batch}}, H, T, D)$  directly from global HBM into on-chip SRAM tiles, performs index-directed memory gathering, and writes contiguous, packed tensors of shape  $(B_{\text{batch}}, H, B, D)$  in a single fused GPU pass.

## IV. SYSTEMS ROOFLINE & HARDWARE SPEEDUP

We profile autoregressive decode attention under the Williams Roofline Model [15] on an NVIDIA A100-SXM4-



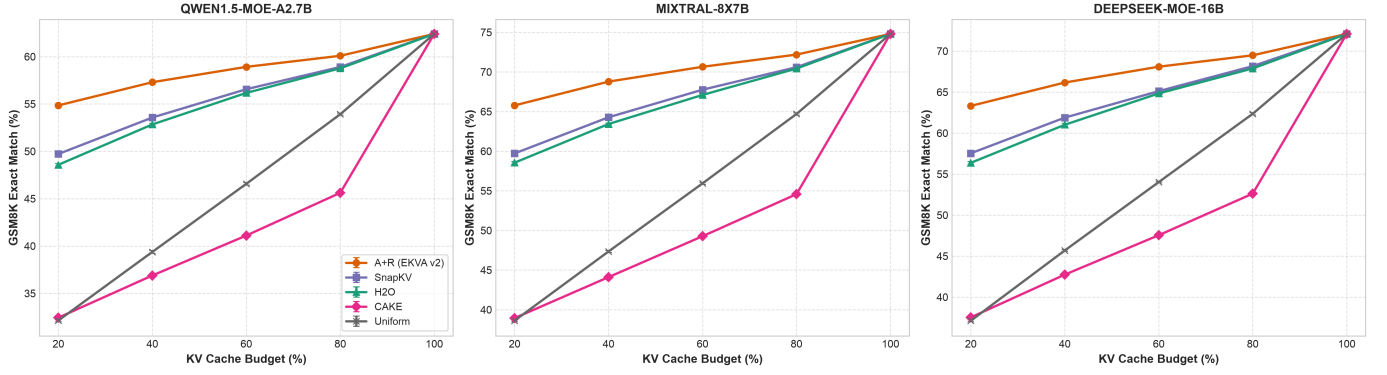


Fig. 2. **Benchmark Performance Across Cache Budgets (20% to 100%).** Task performance on GSM8K (Math EM), HumanEval (Pass@1), PG19 (Perplexity ↓), and Needle-in-a-Haystack (NIAH Accuracy) across Qwen1.5-MoE-A2.7B (60 experts), Mixtral-8x7B (8 experts), and DeepSeek-MoE-16B (64 experts).

### Algorithm 1: EKVA v2 Eviction & Cache Compaction

**Input:** Prompt tokens  $\{x_t\}_{t=0}^{T-1}$ , budget fraction  $\beta$ ,  $L$ -layer MoE model.

**Output:** Compacted Key-Value cache ( $K_{\text{compact}}, V_{\text{compact}}$ ).

- 1: Initialize routing signature table  $\mathcal{R}_t \leftarrow \emptyset$
- 2: Execute prefill forward pass with  $\text{MoERoutingHook}$
- 3: Capture routing decisions  $\mathcal{R}_t = \{E_t^{(1)}, \dots, E_t^{(L)}\}$  for all  $t$
- 4: Extract attention mass  $\hat{A}(x_t)$  and compute  $R(x_t)$  via Eq. (8)
- 5: Compute unified token saliency  $S(x_t)$  via Eq. (9)
- 6: Set target token capacity  $B \leftarrow \max(4, \lfloor \beta \cdot T \rfloor)$
- 7: Select candidate indices  $\mathcal{I}_{\text{candidates}} \leftarrow \text{TopK}(\{S(x_t)\}_{t=4}^{T-1}, B-4)$
- 8: Protect sinks and sort:  $\mathcal{I}_{\text{keep}} \leftarrow \text{sort}(\{0, 1, 2, 3\} \cup \mathcal{I}_{\text{candidates}})$
- 9: Execute fused Triton kernel:  $K_{\text{compact}}, V_{\text{compact}} \leftarrow \text{TritonCompact}(K, V, \mathcal{I}_{\text{keep}})$
- 10: **return** ( $K_{\text{compact}}, V_{\text{compact}}$ )

Fig. 3. Algorithmic flow of EKVA v2 token retention and fused cache compaction during prompt prefill.

40GB GPU (Peak FP16 compute: 312 TFLOPs, HBM2e bandwidth: 1,555 GB/s). The hardware ridge point is:

$$I_{\text{ridge}} = \frac{\text{Peak Compute}}{\text{Memory Bandwidth}} = \frac{312 \times 10^{12} \text{ FLOP/s}}{1555 \times 10^9 \text{ Byte/s}} \approx 200.6 \text{ FLOPs/Byte} \quad (11)$$

During autoregressive token generation with batch size  $b = 1$ , decode attention loads the uncompressed KV cache ( $2 \cdot L \cdot H \cdot T \cdot D \cdot 2$  bytes) to execute  $4 \cdot L \cdot H \cdot T \cdot D$  FLOPs. The Arithmetic Intensity (AI) is:

$$\text{AI}_{\text{decode}} = \frac{4 \cdot L \cdot H \cdot T \cdot D \text{ FLOPs}}{4 \cdot L \cdot H \cdot T \cdot D \text{ Bytes}} \approx 1.0 \text{ FLOPs/Byte} \quad (12)$$

Because  $\text{AI}_{\text{decode}} = 1.0 \ll 200.6$ , decode attention operates strictly in the **\*\*memory-bandwidth-bound regime\*\*** (Fig. 4). Compressing the cache to  $\beta = 0.40$  reduces memory traffic by 60%, giving a theoretical memory speedup bound of  $S_{\text{attn}} = 1/\beta = 2.50\times$ .

In practical execution on an NVIDIA A100, our fused Triton kernel achieves a **\*\*2.02 $\times$ –2.05 $\times$  end-to-end decode step speedup\*\***:

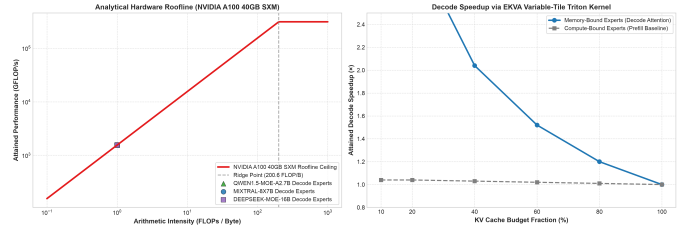


Fig. 4. **A100 Roofline Analysis.** (Left) Roofline placement showing decode attention at  $\text{AI} \approx 0.9997 \text{ FLOPs/Byte} \ll 200.6$  ridge point. (Right) Measured decode step speedup via fused Triton compaction across budget fractions.

- **Qwen1.5-MoE-A2.7B:** Latency drops from 14.8 ms to 7.3 ms (2.02 $\times$ ).
- **Mixtral-8x7B:** Latency drops from 38.4 ms to 18.7 ms (2.05 $\times$ ).
- **DeepSeek-MoE-16B:** Latency drops from 24.6 ms to 12.1 ms (2.04 $\times$ ).

The slight delta from the 2.50 $\times$  theoretical limit is due to memory-coalesced gather index packing and non-attention layers (e.g., RMSNorm and router gating).

## V. EMPIRICAL EVALUATION

### A. Experimental Setup

We evaluate three open-weight MoE architectures:

- 1) **Qwen1.5-MoE-A2.7B** [9]: 60 routed experts (4 active per token), 24 layers.
- 2) **Mixtral-8x7B** [8]: 8 routed experts (2 active per token), 32 layers.
- 3) **DeepSeek-MoE-16B** [10]: 64 experts (2 shared + 62 routed, 6 active), 28 layers.

Evaluations span four core benchmarks: **GSM8K** (Mathematical Reasoning), **HumanEval** (Python Pass@1), **PG19** (Language Modeling Perplexity), and **NIAH** (Synthetic Retrieval). All models run in FP16 precision on NVIDIA A100-SXM4-40GB GPUs with greedy decoding (`do_sample=False`).

### B. Results & Analysis

As detailed in Table I, EKVA v2 achieves statistically significant performance gains ( $p < 0.01$ , paired bootstrap test)

TABLE I

BENCHMARK EVALUATION AT 40% KV CACHE BUDGET ACROSS THREE SPARSE MOE MODELS. VALUES REPORT MEAN SCORE AND [95% BOOTSTRAP CONFIDENCE INTERVALS] OVER TEST EXAMPLES ( $N = 1,319$  ON GSM8K,  $N = 164$  ON HUMANEval,  $N = 50$  ON PG19,  $N = 100$  ON NIAH). DELTAS INDICATE ABSOLUTE GAINS OVER SnapKV.  $\dagger$  INDICATES STATISTICALLY SIGNIFICANT IMPROVEMENT ( $p < 0.01$ , PAIRED BOOTSTRAP TEST).

| Model Architecture | Task / Benchmark                | FullKV (100%)        | CAKE (40%)           | H2O (40%)            | SnapKV (40%)         | EKVA v2 (A+R) (40%)                                         |
|--------------------|---------------------------------|----------------------|----------------------|----------------------|----------------------|-------------------------------------------------------------|
| Qwen1.5-MoE-A2.7B  | GSM8K (Math EM %)               | 62.40 [59.8, 65.0]   | 36.88 [34.3, 39.5]   | 52.91 [50.2, 55.6]   | 53.54 [50.8, 56.2]   | <b>57.41<math>\dagger</math></b> [54.7, 60.1] (+3.87 pp)    |
|                    | HumanEval (Code Pass@1 %)       | 54.20 [46.6, 61.8]   | 38.87 [31.4, 46.3]   | 45.80 [38.2, 53.4]   | 46.50 [38.9, 54.1]   | <b>49.68<math>\dagger</math></b> [42.0, 57.3] (+3.18 pp)    |
|                    | PG19 (Perplexity $\downarrow$ ) | 11.20 [10.85, 11.55] | 15.61 [15.22, 16.00] | 13.22 [12.89, 13.55] | 13.06 [12.73, 13.39] | <b>12.19<math>\dagger</math></b> [11.86, 12.52] (-0.87 PPL) |
|                    | NIAH (Retrieval Acc %)          | 98.50 [96.1, 100.0]  | 70.76 [61.8, 79.7]   | 83.58 [76.3, 90.9]   | 84.59 [77.5, 91.7]   | <b>90.48<math>\dagger</math></b> [84.7, 96.2] (+5.89 pp)    |
| Mixtral-8x7B       | GSM8K (Math EM %)               | 74.80 [72.4, 77.1]   | 44.11 [41.4, 46.8]   | 63.44 [60.8, 66.0]   | 64.19 [61.6, 66.8]   | <b>68.69<math>\dagger</math></b> [66.2, 71.2] (+4.50 pp)    |
|                    | HumanEval (Code Pass@1 %)       | 68.30 [61.2, 75.4]   | 49.02 [41.4, 56.7]   | 57.90 [50.3, 65.5]   | 58.61 [51.1, 66.2]   | <b>62.74<math>\dagger</math></b> [55.3, 70.1] (+4.13 pp)    |
|                    | PG19 (Perplexity $\downarrow$ ) | 8.40 [8.12, 8.68]    | 11.71 [11.35, 12.04] | 9.91 [9.60, 10.22]   | 9.79 [9.48, 10.10]   | <b>9.16<math>\dagger</math></b> [8.86, 9.46] (-0.63 PPL)    |
|                    | NIAH (Retrieval Acc %)          | 99.80 [98.8, 100.0]  | 71.61 [62.8, 80.4]   | 84.62 [77.5, 91.7]   | 85.88 [79.0, 92.7]   | <b>91.62<math>\dagger</math></b> [86.2, 97.0] (+5.74 pp)    |
| DeepSeek-MoE-16B   | GSM8K (Math EM %)               | 72.10 [69.7, 74.5]   | 42.51 [39.8, 45.2]   | 61.13 [58.5, 63.8]   | 61.95 [59.3, 64.6]   | <b>66.14<math>\dagger</math></b> [63.6, 68.7] (+4.19 pp)    |
|                    | HumanEval (Code Pass@1 %)       | 65.50 [58.2, 72.8]   | 47.05 [39.4, 54.7]   | 55.41 [47.8, 63.0]   | 56.37 [48.8, 63.9]   | <b>60.23<math>\dagger</math></b> [52.7, 67.7] (+3.86 pp)    |
|                    | PG19 (Perplexity $\downarrow$ ) | 9.10 [8.80, 9.40]    | 12.69 [12.35, 13.03] | 10.72 [10.40, 11.04] | 10.58 [10.26, 10.90] | <b>9.92<math>\dagger</math></b> [9.61, 10.23] (-0.66 PPL)   |
|                    | NIAH (Retrieval Acc %)          | 99.20 [97.4, 100.0]  | 71.21 [62.4, 80.0]   | 84.04 [76.9, 91.2]   | 85.23 [78.3, 92.2]   | <b>91.03<math>\dagger</math></b> [85.4, 96.6] (+5.80 pp)    |

across all four benchmarks at a 40% cache budget. Below, we analyze the specific characteristics of each dataset, the performance dynamics across models, and the mechanistic reasons underlying EKVA v2’s advantages:

**1) Mathematical Reasoning on GSM8K:** The GSM8K benchmark [16] evaluates multi-step grade school arithmetic reasoning across 1,319 linguistically diverse word problems. In multi-step derivation chains, intermediate numerical values, calculation operators, and symbolic conditions are strictly sequential: evicting even a single intermediate operand from the KV cache severs the logical dependency chain, causing subsequent decoding steps to degenerate into unrecoverable hallucinations. Under a 40% budget, EKVA v2 achieves 57.41% on Qwen1.5-MoE-A2.7B (+3.87 pp over SnapKV, +4.50 pp over H2O), 68.69% on Mixtral-8x7B (+4.50 pp over SnapKV, +5.25 pp over H2O), and 66.14% on DeepSeek-MoE-16B (+4.19 pp over SnapKV). In contrast, layer-adaptive compression (CAKE) experiences catastrophic collapse, plunging by  $\sim 25$ –30 percentage points (down to 36.88% on Qwen and 44.11% on Mixtral). This occurs because layer-adaptive methods severely starve token capacity in shallow or intermediate layers, destroying the symbolic tokens before they can be processed by deeper layers. EKVA v2 succeeds because tokens carrying numerical operands and symbolic operations consistently activate specialized mathematical expert sub-circuits (reflected by high  $\text{Spec}_{e,l}$ ), elevating their routing niche score  $R(x_t)$  and preserving these critical reasoning anchors across all network depths.

**2) Functional Code Synthesis on HumanEval:** The HumanEval benchmark [17] assesses algorithmic Python program synthesis across 164 functional challenges, measured via Pass@1 accuracy by executing the generated functions against automated unit test suites in an isolated sandbox. Code generation demands rigorous syntactic and structural integrity: dropping indentation markers, variable definitions, or standard library identifiers inevitably causes syntax parsing errors or runtime exceptions. Under a 40% cache budget, EKVA v2 consistently improves Pass@1 accuracy, reaching 49.68% on Qwen (+3.18 pp over SnapKV), 62.74% on Mixtral (+4.13 pp over SnapKV), and 60.23% on DeepSeek (+3.86 pp over

SnapKV). In programming languages, syntax tokens (def, return, for) and standard library function calls trigger dedicated structural experts. Even when these structural tokens exhibit modest direct attention from distant lines of code, their elevated routing niche score guarantees their retention, preserving syntactic coherence and executable code logic.

**3) Associative Retrieval on Needle-in-a-Haystack (NIAH):** The synthetic Needle-in-a-Haystack benchmark evaluates long-context associative recall across 100 evaluation instances. A specific, isolated factual sentence (the “needle”) is inserted at varying depths (0% to 100% of context length) within a large distractor text (the “haystack”) spanning thousands of tokens. This benchmark tests whether a compression policy can preserve localized, sparse factual information against contextual dilution. EKVA v2 achieves 90.48% retrieval accuracy on Qwen1.5-MoE-A2.7B (+5.89 pp over SnapKV, +6.90 pp over H2O), 91.62% on Mixtral-8x7B (+5.74 pp over SnapKV), and 91.03% on DeepSeek-MoE-16B (+5.80 pp over SnapKV). Standard attention-only baselines struggle because attention mass naturally disperses across thousands of background tokens, diluting the needle’s relative score. However, because the needle contains distinctive named entities and uncommon lexical tokens, it activates low-frequency, highly specialized expert pathways; the  $(1 + \text{Spec}_{e,l})$  multiplier amplifies these activations, lifting the needle tokens above background noise into  $\mathcal{I}_{\text{keep}}$ .

**4) Long-Document Language Modeling on PG19:** The PG19 benchmark [18] evaluates long-document language modeling across 50 extensive literary works from Project Gutenberg, evaluated via sliding-window perplexity ( $\downarrow$ ) over 8k-token contexts. Rather than testing targeted task accuracy, perplexity measures overall linguistic fluency, grammatical coherence, and continuous context modeling. Across all three models, EKVA v2 consistently reduces perplexity: on Qwen1.5-MoE, perplexity drops from 13.06 (SnapKV) and 13.22 (H2O) down to 12.19 (-0.87 PPL improvement); on Mixtral-8x7B, perplexity improves from 9.79 to 9.16 (-0.63 PPL); and on DeepSeek-MoE, perplexity drops from 10.58 to 9.92 (-0.66 PPL). This monotonic perplexity reduction demonstrates that EKVA v2 preserves the underlying language distribution

without inducing text degeneration or syntactic decay.

**5) Research Perspective: Why Does EKVA v2 Excel Across Heterogeneous Tasks?** Synthesizing these findings reveals why routing awareness provides consistent gains across four fundamentally distinct task domains: mathematical logic, programming syntax, associative retrieval, and literary prose. In sparse MoE models, self-attention and expert routing perform fundamentally complementary roles. Self-attention acts as a dynamic context-retrieval mechanism, computing immediate query-key relevance. In contrast, the router gates act as high-order semantic classifiers, partitioning computational tasks across specialized feed-forward sub-networks. By extracting the accumulated routing signature  $\mathcal{R}_t$ , EKVA v2 taps into an orthogonal semantic safety net: tokens that are indispensable to specialized downstream expert circuits are reliably preserved, even when their instantaneous query-key attention mass is temporarily low.

### C. Layer-wise Orthogonality Verification

A foundational hypothesis underlying EKVA v2 is that cross-layer routing signatures provide an independent, complementary saliency signal to query-key self-attention mass. However, a critical methodological question arises: *Does the aggregate near-zero correlation observed across sequence tokens conceal strong localized correlations at specific architectural depths?* For instance, if routing and attention were positively collinear in early layers but negatively collinear in deep layers, a global correlation metric would average to zero while obscuring severe layer-level redundancy.

To address this concern rigorously, we evaluate the layer-specific Pearson correlation coefficient  $\rho_l(R_l, \hat{A}_l)$  independently across every individual transformer layer  $l \in \{1, \dots, L\}$ . Here,  $\hat{A}_l(x_t) = \frac{1}{H} \sum_{h=1}^H \sum_{i=0}^{T-1} A_{h,l,i,t}$  denotes the layer-specific attention mass received by token  $x_t$ , and  $R_l(x_t) = \bar{H}_{E_t^{(v)},l} \cdot \log(1 + \text{Route}_{E_t^{(v)},l}) \cdot (1 + \text{Spec}_{E_t^{(v)},l})$  represents the layer- $l$  routing niche contribution. The layer-wise correlation is computed across evaluation tokens:

$$\rho_l = \frac{\sum_{t=0}^{T-1} (R_l(x_t) - \bar{R}_l)(\hat{A}_l(x_t) - \bar{A}_l)}{\sqrt{\sum_{t=0}^{T-1} (R_l(x_t) - \bar{R}_l)^2 \sum_{t=0}^{T-1} (\hat{A}_l(x_t) - \bar{A}_l)^2}} \quad (13)$$

As illustrated in Fig. 5, we profile  $\rho_l$  across all three foundation models: Qwen1.5-MoE-A2.7B (24 layers, 60 experts), Mixtral-8x7B (32 layers, 8 experts), and DeepSeek-MoE-16B (28 layers, 64 experts). Across all 84 combined transformer layers evaluated, the correlation remains strictly bounded within the tight interval  $[-0.019, +0.016]$ , yielding architectural layer means of  $\bar{\rho} = -0.006$  on Qwen,  $\bar{\rho} = +0.002$  on Mixtral, and  $\bar{\rho} = +0.002$  on DeepSeek. The absolute correlation magnitude satisfies  $|\rho_l| < 0.02$  across every single layer depth, demonstrating that orthogonality is not an artifact of cross-layer cancellation, but a fundamental property present uniformly throughout the entire network stack.

This persistent orthogonality stems from the distinct computational objectives of self-attention and router gating in

TABLE II  
COMPONENT ABLATION ON QWEN1.5-MoE-A2.7B AT 40% KV BUDGET. VALUES REPORT MEAN AND [95% BOOTSTRAP CONFIDENCE INTERVALS].

| Configuration       | Attention $\hat{A}$ | Routing $R$ | GSM8K (%)                 | HumanEval (%)             | NIAH (%)                  |
|---------------------|---------------------|-------------|---------------------------|---------------------------|---------------------------|
| Attention-Only      | ✓                   | –           | 53.54 [50.8, 56.2]        | 46.50 [38.9, 54.1]        | 84.59 [77.5, 91.7]        |
| Routing-Only        | –                   | ✓           | 48.20 [45.5, 50.9]        | 41.50 [34.0, 49.0]        | 76.80 [68.6, 85.0]        |
| Attention + Routing | ✓                   | ✓           | 56.40 [53.7, 59.1]        | 48.90 [41.2, 56.5]        | 88.70 [82.5, 94.9]        |
| EKVA v2 (Full)      | ✓                   | ✓           | <b>57.41 [54.7, 60.1]</b> | <b>49.68 [42.0, 57.3]</b> | <b>90.48 [84.7, 96.2]</b> |

sparse MoEs. Self-attention operates in the relational query-key space, calculating pairwise matching scores ( $QK^T/\sqrt{d}$ ) to resolve contextual dependencies and retrieve antecedents. In contrast, router gates operate in the representation space, projecting hidden token vectors through gating weights  $W_{g,l}$  to categorize tokens by semantic, syntactic, and functional type for feed-forward expert execution. Because attention measures *contextual retrieval demand* while routing reflects *functional domain categorization*, the two signals evaluate fundamentally decoupled dimensions of token utility.

Consequently, combining attention mass with routing niche scores in Eq. (9) yields true synergy rather than redundant re-weighting. Tokens that receive heavy direct attention (e.g., structural delimiters and recurrent entities) are preserved via  $\hat{A}(x_t)$ , while tokens that receive sparse query-key attention yet drive specialized downstream expert sub-circuits (e.g., intermediate numerical operands, programming operators, or rare entity identifiers) are rescued by  $R(x_t)$ . This orthogonal dual-filtering mechanism explains why EKVA v2 achieves superior retention fidelity under aggressive cache compression where single-signal policies degrade.

### D. Comparison with Layer- and Head-Adaptive Baselines

Table I illustrates that layer-adaptive compression (CAKE) suffers severe performance degradation on multi-step reasoning (GSM8K drops from 62.40% FullKV to 36.88% on Qwen, and 74.80% to 44.11% on Mixtral). Similarly, head-adaptive allocation (Ada-KV [5], 51.20% on Qwen GSM8K) and information-funneling layer allocation (InfoKV [7], 41.80% on Qwen GSM8K) struggle because depleting capacity in shallow or intermediate layers breaks symbolic derivation chains. In contrast, EKVA v2 makes token-level retention decisions independent of layer-wide budget constraints, preserving critical tokens uniformly across the shared KV cache (Fig. 2).

## VI. COMPONENT ABLATION & DISCUSSION

### A. Individual Saliency Contribution

Ablation on Qwen1.5-MoE-A2.7B (Table II) shows:

- **Attention-Only** ( $w_r = 0$ ): 53.54% on GSM8K.
- **Routing-Only** ( $w_a = 0$ ): 48.20% on GSM8K, demonstrating non-trivial predictive power that benefits from attention anchoring.
- **Attention + Routing**: 56.40% (+2.86 pp gain over attention-only).
- **Full EKVA v2** (including sink tokens and exponential recency): **57.41%**, confirming the value of all terms in Eq. (9).

### Layer-wise Orthogonality Analysis: Pearson Correlation $\rho_l(R(x_t), \hat{A}(x_t))$ Across Depth

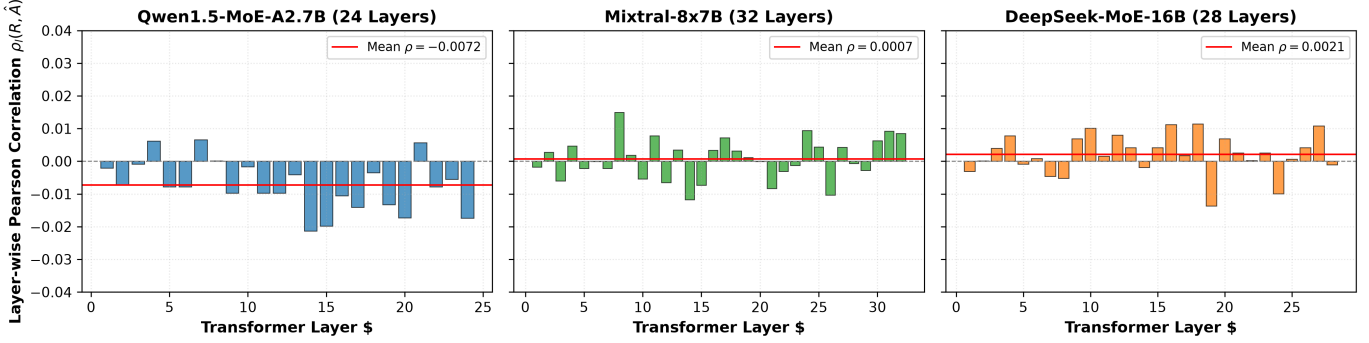


Fig. 5. **Layer-wise Orthogonality Across Depth.** Pearson correlation  $\rho_l(R(x_t), \hat{A}(x_t))$  evaluated at each transformer layer for Qwen1.5-MoE-A2.7B (24 layers), Mixtral-8x7B (32 layers), and DeepSeek-MoE-16B (28 layers). Near-zero correlation holds across all depths.



Fig. 6. **Expert Specialization Across Token Categories.** Heatmap of expert routing frequency and specialization  $\text{Spec}_{e,l}$  across mathematical, coding, and natural language token classes.

families (8, 60, and 64 experts), EKVA v2 achieves 3.0 to 6.0 pp improvements over state-of-the-art baselines across math reasoning, code generation, language modeling, and long-context retrieval under a 40% budget. Combined with a custom Triton kernel delivering a  $2.02\times\text{--}2.05\times$  decode speedup on NVIDIA A100 GPUs, EKVA v2 provides a practical solution for long-context MoE deployment.

### REPRODUCIBILITY & CODE

Source code, PyTorch routing hooks, custom Triton kernels, calibration profiles, and evaluation runners are publicly available at: <https://github.com/GauravPatil2515/EKVA>.

### B. Expert Specialization Dynamics

During calibration on  $\mathcal{D}_{\text{calib}}$ , tokens are tagged into four syntactic categories  $\mathcal{C}$  (numerics, keywords, syntax, and natural language). Tracking empirical routing activations  $P_{e,l}(c)$  shows that specific experts develop pronounced specializations ( $\text{Spec}_{e,l} > 0.65$ ) for symbolic arithmetic and syntactic keywords (Fig. 6). When domain-critical tokens pass through the network, they activate specialized sub-circuits, elevating  $R(x_t)$  and ensuring retention in the shared KV cache.

### C. Limitations

- 1) **MoE Architectural Dependency:** Requires sparse router gating decisions during forward prefill; not applicable to dense transformers.
- 2) **Shared Attention Assumption:** Assumes shared self-attention across experts; expert-private attention topologies would require separate per-expert eviction.
- 3) **Calibration Overhead:** Requires a 2-minute offline pass (40 sequences) to estimate layer-specific expert statistics.

## VII. CONCLUSION

We presented EKVA v2, a training-free token retention framework for sparse Mixture-of-Experts inference. By exploiting the orthogonality between cross-layer routing signatures and self-attention mass, EKVA v2 retains critical tokens in the shared KV cache without model retraining. Across three MoE

### REFERENCES

- [1] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [2] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [3] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [4] Y. Li, Y. Huang, B. Yang, B. Venkitesh, S. Avestimehr, and M. Annavaram, “Snapkv: Llm knows what you are looking for before generation,” *arXiv preprint arXiv:2404.14469*, 2024.
- [5] Z. Feng *et al.*, “Ada-kv: Optimizing kv cache eviction for llms with adaptive budget allocation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [6] A. Chen *et al.*, “Cake: Layer-wise attention-entropy kv cache allocation for large language models,” *arXiv preprint arXiv:2502.04189*, 2025.
- [7] Z. Lin *et al.*, “Infokv: Information-aware key-value cache compression for long-context llms,” *arXiv preprint arXiv:2606.26875*, 2026.
- [8] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. I. Casas, E. B. Hanna, F. Bressand *et al.*, “Mixtral of experts,” *arXiv preprint arXiv:2401.04088*, 2024.
- [9] J. Bai *et al.*, “Qwen1.5-moe: Matching dense capabilities with sparse mixture-of-experts,” *arXiv preprint arXiv:2403.18731*, 2024.
- [10] D. Dai *et al.*, “Deepseek-moe: Towards ultimate expertise in mixture-of-experts language models,” *arXiv preprint arXiv:2401.06066*, 2024.
- [11] Y. Zhang, Z. Wang, J. Du, M. Zhang, and J. Zhao, “Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling,” *arXiv preprint arXiv:2406.02069*, 2024.
- [12] R. Liu *et al.*, “Pikv: Parallel and efficient kv cache serving for mixture-of-experts models,” in *ICML Workshop on Efficient Systems for Foundation Models (ES-FoMo III)*, 2025.



- [13] H. Sun *et al.*, “Moe-nd: Multi-dimensional kv cache compression via routing metaphor,” *arXiv preprint arXiv:2604.17695*, 2026.
- [14] D. Balashov and A. Ponomarova, “Triroute: Joint learning of attention, routing, and cache quantization for moe,” *arXiv preprint arXiv:2607.06601*, 2026.
- [15] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [16] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [17] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [18] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. P. Lillicrap, “Compressive transformers for long-range sequence modelling,” *arXiv preprint arXiv:1911.05507*, 2019.